



PROGRAMA DE FORMACIÓN MARVELL

Diseño Digital para Sistemas DSP

Ejercicios Módulo 3: Codificación Numérica

Profesor: Del Barco, Martín.

Alumnos: Monja, Ernesto Joaquín
Zúñiga, Guillermo Rubén Darío

Índice

1. Introducción	2
2. Desarrollo	2
2.1. Conversión Float $\longleftrightarrow$ Fixed	2
2.2. Suma en Punto Fijo	3
2.3. Truncado, Redondeo y Saturación	4
2.4. CSD — Recodificación	6
2.5. Booth radix-2	8
2.6. RNS — Multiplicación Modular	9
3. Conclusiones	10
4. Referencias	11

1. Introducción

El presente informe documenta la resolución práctica de los ejercicios del Módulo 3: Codificación Numérica, cuyo objetivo es afianzar la representación de datos y el diseño de arquitecturas aritméticas en *HW*. A lo largo del trabajo se abordan la conversión entre formatos numéricos, la aritmética en punto fijo con técnicas de recorte (truncado, redondeo y saturación), estrategias de optimización como la re-codificación *CSD* y el algoritmo de *Booth radix-2*, y la multiplicación modular *RNS*. Para garantizar la correctitud de los diseños, se implementó un flujo de verificación que contrasta modelos teóricos desarrollados en *Python* (vía `fxpmath`) con simulaciones de los módulos en *Verilog*, evaluando su comportamiento temporal mediante *Icarus Verilog* y *GTKWave*.

2. Desarrollo

2.1. Conversión Float $\longleftrightarrow$ Fixed

En este ejercicio se requiere realizar la conversión del número decimal $x = +9,625$ a dos formatos distintos utilizados en arquitecturas *DSP*: *IEEE 754* de simple precisión y punto fijo signado.

Como paso preliminar, se convierte el valor decimal a binario puro, tal que la parte entera es $9_{10} = 1001_2$, y la parte fraccionaria esta dada según la [Ec. 1](#).

$$0,625_{10} = 0,5_{10} + 0,125_{10} = 2^{-1} + 2^{-3} = 0,101_2 \quad (1)$$

Por lo tanto, el número completo en binario es:

$$x = 1001,101_2 \quad (2)$$

a) Representación en *IEEE-754* simple precisión (32 bits)

El estándar emplea 1 bit de signo (S), 8 bits de exponente (E) y 23 bits de mantisa (M), por lo tanto se tiene que siguiendo este estándar:

- Signo: Al ser un número positivo, el bit de signo es $S = 0$.
- Exponente: Se normaliza el valor binario desplazando la coma hacia la izquierda hasta dejar un único "1.^{en} la parte entera, tal que:

$$1001,101_2 = 1,001101_2 \times 2^3 \quad (3)$$

Se tiene entonces que según [Ec. 3](#), el exponente real es 3. Aplicando el sesgo de 127

correspondiente a la simple precisión, se obtiene:

$$E = 3_{10} + 127_{10} = 130_{10} = 10000010_2 \quad (4)$$

- Mantisa: Se toman los bits fraccionales del número normalizado (001101_2) y se rellena con ceros a la derecha hasta completar los 23 bits lo que resulta en:

$$M = 00110100000000000000000_2 \quad (5)$$

El resultado final explícito de 32 bits es:

$$+9,625_{10} = 0 \ 10000010 \ 00110100000000000000000_2 \quad (6)$$

b) Representación en punto fijo S(8, 3)

El formato S(8, 3) indica un número signado en complemento a 2 con una longitud total de 8 bits, de los cuales 3 están destinados a la parte fraccional. Esto deja 1 bit para el signo y 4 bits para la magnitud entera.

Dado que el número original $1001,101_2$ posee exactamente 4 bits enteros y 3 bits fraccionales, encaja de manera directa en el formato sin requerir truncamiento:

$$S(8, 3)_{+9,625} = 01001101 \quad (7)$$

c) Error de cuantización

El error de cuantización se produce cuando la cantidad de bits fraccionales del formato destino no es suficiente para representar la precisión del número original. En este caso, la parte fraccionaria de $x = 9,625$ requiere exactamente 3 bits ($0,101_2$). Como el formato elegido es S(8, 3), no existe pérdida de información y por lo tanto se tiene que:

- Error en LSB: $e_{LSB} = 0$
- Error en valor absoluto: $e_{abs} = 0$.

2.2. Suma en Punto Fijo

Para esta sección se calculó la suma de dos valores en punto fijo. Estos valores son:

- $A = -1,75$ en S(6,4)
- $B = 0,9375$ en S(8,5)

Para obtener el resultado S , se deben alinear los operandos para tener la coma implícita en la misma posición, luego se expanden los bits fraccionales y el signo del operando con

menor cantidad de bits. Así, se tiene una suma de dos valores con el mismo formato, pero con un bit más en el resultado para abarcar el carry.

A		1	1	0	0	1	0	0	0
B		0	0	0	1	1	1	1	0
S		1	1	1	1	0	0	1	1

Cuadro 1: Suma de A y B, expandiendo el formato de A.

Se realizó el código de esta suma en *Python* y *Verilog*. El resultado en *Python* imprime lo mostrado en la Fig. 1, coincidiendo la suma en binario con la suma en punto flotante y en decimal.

A:	10.0100		-1.75		-28
B:	000.11110		0.9375		30
A:	110.01000		-1.75		-56
B:	000.11110		0.9375		30

S:	1111.00110		-0.8125		-26

Figura 1: Salida del script de suma en Python.

Por otro lado, el resultado del testbench de la suma en *Verilog* se muestra en la Fig. 2. observándose entonces que el resultado de los dos desarrollos concuerdan con el resultado esperado en forma teórica.

```
>>> Ejecutando con vvp...
VCD info: dumpfile tb_suma.vcd opened for output.
A: 100100 | -28
B: 00011110 | 30

A: 11001000 | -56
B: 00011110 | 30
-----
S: 111100110 | -26
tb_suma.sv:51: $finish called at 10000 (1ps)
VCD generado. Abrir con: gtkwave tb_suma.vcd
```

Figura 2: Impresión del testbench de suma.

2.3. Truncado, Redondeo y Saturación

En esta sección se aborda la implementación y verificación en *HW* de las técnicas de recorte (truncamiento, redondeo y saturación) para señales en formato de punto fijo. Como paso inicial, se utilizó la librería *fxpmath* de *Python* para generar un *benchmark* que permita contrastar los resultados de la simulación.

Para el análisis se plantearon los siguientes casos:

- Conversión de la señal $x = 5,5625$ del formato $S(10, 6)$ al formato $S(7, 3)$ aplicando truncamiento y redondeo al más cercano.
- Conversión de la señal $y = 8,75$ del formato $S(11, 6)$ al formato $S(5, 3)$ aplicando *wrap-around* y saturación. Cabe destacar que se modificó el formato original dictado por la consigna para la señal y (de $S(10, 6)$ a $S(11, 6)$) añadiendo un bit a la parte entera, con el fin de evitar un desbordamiento (*overflow*) en la señal de entrada.

En la Fig. 3 se observa la salida de la terminal, detallando los resultados teóricos esperados generados por el script de referencia.

```
>>> Generando benchmark con Python (fxpmath)...
--- Cálculos de referencia con fxpmath ---
x_in (S10.6): 0101100100 -> 5.5625
x_trunc (S7.3): 0101100 -> 5.5
x_round (S7.3): 0101100 -> 5.5

y_in (S11.6): 01000110000 -> 8.75
y_wrap (S5.3): 00110 -> 0.75
y_sat (S5.3): 01111 -> 1.875

>>> Compilando con iverilog...
>>> Ejecutando con vvp...
VCD info: dumpfile recorte.vcd opened for output.
Simulación terminada. Revisar el VCD en GTKWave.
tb_recorte.v:42: $finish called at 30000 (1ps)
```

Figura 3: Resultados del benchmark en *Python* y ejecución del entorno.

Por otro lado, dado que se programo tanto un modulo como un testbench en *Verilog* se propone graficar estos cambios utilizando *GTKWave* tal como se muestra en la Fig. 4.

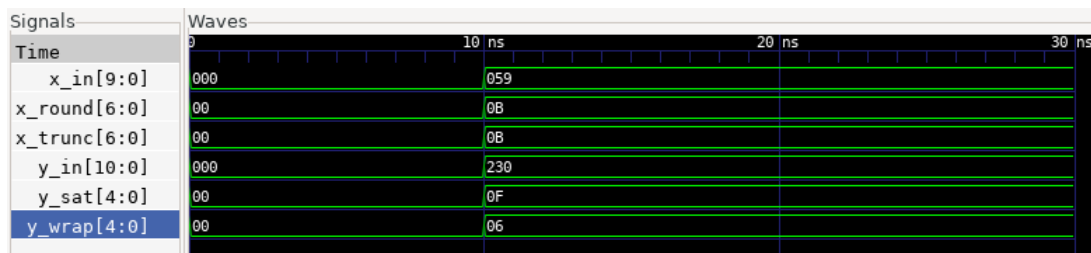


Figura 4: Simulación en GTKWave mostrando las señales de entrada y salida en formato hexadecimal.

Al analizar las formas de onda, se corroboran los siguientes resultados:

- Señal x : La entrada asume el valor 059_h (en hexadecimal) ($5,5625_{10}$ en base decimal). Tras el recorte, tanto el truncamiento (x_trunc) como el redondeo (x_round) arrojan el valor $0B$ ($5,5_{10}$).
- Señal y : La entrada asume el valor 230_h (en hexadecimal) ($8,75_{10}$). Al aplicar un recorte directo por *wrap-around*, se pierden los bits más significativos y el valor

resultante cae a 06 ($0,75_{10}$). Por el contrario, al aplicar lógica de saturación, el *HW* detecta el *overflow* positivo y limita la salida al máximo valor representable en el formato $S(5, 3)$, que corresponde a 0F ($1,875_{10}$).

Los resultados obtenidos en la simulación temporal coinciden con total exactitud con el modelo de referencia.

2.4. CSD — Recodificación

La representación *Canonical Signed Digit* (CSD) es un método utilizado en *DSP* que ofrece una optimización en la cantidad de sumadores cuando se requiere hacer una multiplicación por una constante, en algunos casos. Esto conlleva a disminuir el área y la potencia, pero la constante debe tener varios unos consecutivos para obtener una mejora significativa.

Se realizó la multiplicación de una señal variable y signada con el valor $K = 23$, en binario estándar y en CSD. La conversión de K a *CSD* se muestra en la Tab. 2. De esta forma, se obtiene un coeficiente sin no-ceros consecutivos y se espera que la cantidad de sumas realizadas en una multiplicación pasen de 3 a 2.

Binario	0	1	0	1	1	1
	0	1	1	0	0	-1
CSD	1	0	-1	0	0	-1

Cuadro 2: Conversión del valor K a *CSD*.

Realizando la multiplicación de K por una señal $S(4, 0)$ en *Python*, se obtiene el resultado de la Fig. 5. Si se observa el código de la multiplicación en la Fig. 6, se puede ver que se redujo la cantidad de sumadores como se esperó, ya que se usaron 3 sumas en el caso del binario estándar y 2 sumas en el caso del *CSD*.

La implementación del módulo en *Verilog* se realizó con el mismo algoritmo y el testbench arrojó los mismos resultados que el algoritmo de *Python*. Estos se muestran en la Fig. 7. Nuevamente, en el *RTL* se comprueba la reducción de sumadores, como se muestra en la Fig. 8.

```

K Estandar: 0 1 0 1 1 1
K CSD: 1 0 -1 0 0 -1
X: 0000 ( 0 ) | Y (est): 000000000 ( 0.0 ) | Y (csd): 0000000000 ( 0.0 )
X: 0001 ( 1 ) | Y (est): 000010111 ( 23.0 ) | Y (csd): 0000010111 ( 23.0 )
X: 0010 ( 2 ) | Y (est): 000101110 ( 46.0 ) | Y (csd): 0000101110 ( 46.0 )
X: 0011 ( 3 ) | Y (est): 001000101 ( 69.0 ) | Y (csd): 0001000101 ( 69.0 )
X: 0100 ( 4 ) | Y (est): 001011100 ( 92.0 ) | Y (csd): 0001011100 ( 92.0 )
X: 0101 ( 5 ) | Y (est): 001110011 ( 115.0 ) | Y (csd): 0001110011 ( 115.0 )
X: 0110 ( 6 ) | Y (est): 010001010 ( 138.0 ) | Y (csd): 0010001010 ( 138.0 )
X: 0111 ( 7 ) | Y (est): 010100001 ( 161.0 ) | Y (csd): 0010100001 ( 161.0 )
X: 1000 (-8 ) | Y (est): 101001000 ( -184.0 ) | Y (csd): 1101001000 ( -184.0 )
X: 1001 (-7 ) | Y (est): 101011111 ( -161.0 ) | Y (csd): 1101011111 ( -161.0 )
X: 1010 (-6 ) | Y (est): 101110110 ( -138.0 ) | Y (csd): 1101110110 ( -138.0 )
X: 1011 (-5 ) | Y (est): 110001101 ( -115.0 ) | Y (csd): 1110001101 ( -115.0 )
X: 1100 (-4 ) | Y (est): 110100100 ( -92.0 ) | Y (csd): 1110100100 ( -92.0 )
X: 1101 (-3 ) | Y (est): 110111011 ( -69.0 ) | Y (csd): 1110111011 ( -69.0 )
X: 1110 (-2 ) | Y (est): 111010010 ( -46.0 ) | Y (csd): 1111010010 ( -46.0 )
X: 1111 (-1 ) | Y (est): 111101001 ( -23.0 ) | Y (csd): 1111101001 ( -23.0 )

```

Figura 5: Resultado impreso de la multiplicación en binario estándar y con CSD en Python.

```

# Estandar
Y0((X[i] << 4) + (X[i] << 2) + (X[i] << 1) + X[i]) ())
# CSD
Y1((X[i] << 5) - (X[i] << 3) - X[i]) ())

```

Figura 6: Asignación de la multiplicación en Python.

```

>>> Compilando con iverilog...
>>> Ejecutando con vvp...
x: 0000 (0) | y (est): 000000000 (0) | y (csd): 0000000000 (0)
x: 0001 (1) | y (est): 000010111 (23) | y (csd): 0000010111 (23)
x: 0010 (2) | y (est): 000101110 (46) | y (csd): 0000101110 (46)
x: 0011 (3) | y (est): 001000101 (69) | y (csd): 0001000101 (69)
x: 0100 (4) | y (est): 001011100 (92) | y (csd): 0001011100 (92)
x: 0101 (5) | y (est): 001110011 (115) | y (csd): 0001110011 (115)
x: 0110 (6) | y (est): 010001010 (138) | y (csd): 0010001010 (138)
x: 0111 (7) | y (est): 010100001 (161) | y (csd): 0010100001 (161)
x: 1000 (-8) | y (est): 101001000 (-184) | y (csd): 1101001000 (-184)
x: 1001 (-7) | y (est): 101011111 (-161) | y (csd): 1101011111 (-161)
x: 1010 (-6) | y (est): 101110110 (-138) | y (csd): 1101110110 (-138)
x: 1011 (-5) | y (est): 110001101 (-115) | y (csd): 1110001101 (-115)
x: 1100 (-4) | y (est): 110100100 (-92) | y (csd): 1110100100 (-92)
x: 1101 (-3) | y (est): 110111011 (-69) | y (csd): 1110111011 (-69)
x: 1110 (-2) | y (est): 111010010 (-46) | y (csd): 1111010010 (-46)
x: 1111 (-1) | y (est): 111101001 (-23) | y (csd): 1111101001 (-23)
tb_csd.sv:40: $finish called at 160000 (1ps)

```

Figura 7: Resultado del testbench, en binario estándar y con CSD en Verilog.

```

assign y_est = (x <<< 4) + (x <<< 2) + (x <<< 1) + x;
assign y_csd = (x <<< 5) - (x <<< 3) - x;

```

Figura 8: Asignación de la multiplicación en Verilog.

2.5. Booth radix-2

En este ejercicio se implementó un multiplicador por *HW* utilizando el algoritmo de *Booth radix-2*. El objetivo consistió en realizar el producto de dos números signados de 4 bits representados en complemento a 2: el multiplicando $A = +6$ y el multiplicador $B = -5$.

Para aplicar el algoritmo, se extendió el multiplicador B agregando un bit virtual $b_{-1} = 0$ en la posición menos significativa, permitiendo evaluar los pares de bits (b_i, b_{i-1}) para determinar las sumas y restas de los productos parciales desplazados.

Como paso preliminar, se generó un modelo de referencia en Python utilizando la librería `fxpmath`, cuyos resultados por consola se detallan en la [Fig. 9](#).

```
>>> Generando benchmark con Python (fxpmath)...
--- Cálculos de referencia con fxpmath ---
A = 6 -> 0110
B = -5 -> 1011
P = A * B = -30 -> 11100010

>>> Compilando con iverilog...
>>> Ejecutando con vvp...
VCD info: dumpfile booth.vcd opened for output.
Simulación terminada. Revisa el VCD en GTKWave.
tb_booth.v:34: $finish called at 30000 (1ps)

>>> Abriendo las ondas en GTKWave...

GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
[30000] end time.
```

Figura 9: Resultados del benchmark en *Python* y ejecución de la compilación.

El script de referencia calculó que el producto de dos señales de 4 bits genera un resultado de 8 bits. Posteriormente, se simuló el *HW* diseñado en *Verilog* y se visualizaron los resultados temporales mediante *GTKWave*, como se muestra en la [Fig. 10](#).

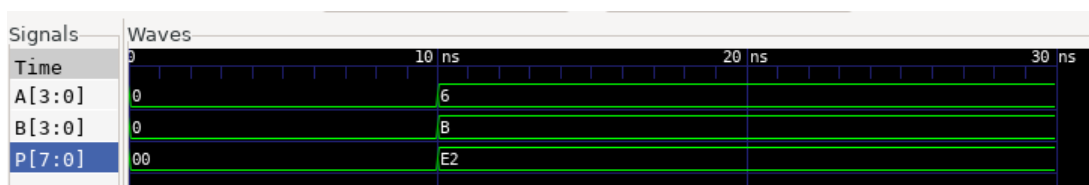


Figura 10: Simulación en GTKWave del multiplicador Booth radix-2.

Al analizar las formas de onda de la [Fig. 10](#), se verifica la correcta operación del módulo:

- Multiplicando (A): El valor de entrada es 6 en hexadecimal, correspondiente a 0110_2 o $+6_{10}$ en complemento a 2.
- Multiplicador (B): El valor de entrada es B en hexadecimal, correspondiente a 1011_2 o -5_{10} en complemento a 2.
- Producto (P): La salida obtenida es E2 en hexadecimal. Su representación binaria es 11100010_2 , lo que equivale exactamente a -30_{10} en complemento a 2.

De esta manera, se comprueba que el comportamiento del *HW* diseñado coincide con la verificación teórica $A \cdot B = -30$ establecida en la consigna.

2.6. RNS — Multiplicación Modular

El sistema de representación *Residue Number System* (RNS) consiste en representar un número entero por sus residuos respecto a valores coprimos, resultando en varias operaciones más pequeñas en paralelo y sin propagación de carry. Esto puede aumentar el área pero permite un paralelismo muy alto.

Tomando dos ejemplos, se representaron dos valores X e Y en *RNS*, tomando el conjunto de módulos $\{3, 5, 7\}$, y luego se realizó el producto en cada módulo. El desarrollo en forma teórica se muestra en el [Tab. 3](#).

	Mod. 3	Mod. 5	Mod. 7
$X = 14$	2	4	0
$Y = 6$	0	1	6
$P = 84$	0	4	0

Cuadro 3: Multiplicación modular entre X e Y.

En *Python* se armó un vector de valores modulares para cada variable y se realizó el producto modular. El resultado se puede ver en la [Fig. 11](#).

```
X: 14 | [2, 4, 0]
Y: 6  | [0, 1, 6]
P: 84 | [0, 4, 0]
84 % 3 = 0
84 % 5 = 4
84 % 7 = 0
```

Figura 11: Resultado del producto modular en Python.

Mientras tanto, en *Verilog* no se utilizó un arreglo de valores modulares, sino que se usaron 3 variables internas para cada señal, cada una con distinto número de bits para optimizar el área. Para la multiplicación modular, se utilizó el operador “%”, que no es sintetizable pero es útil para observar este caso en particular. El resultado puede observarse en la [Fig. 12](#) y se comprueba el resultado correcto en comparación con el teórico.

```
>>> Compilando con iverilog...
>>> Ejecutando con vvp...
X: 14 | [2, 4, 0]
Y:  6 | [0, 1, 6]
P:    | [0, 4, 0]
```

Figura 12: Resultado del producto modular en Verilog.

3. Conclusiones

Este trabajo presenta la implementación de varias formas de representación y calculo numérico en un *DSP*, desarrolladas tanto en *Python* como en *Verilog*. Se realizó la conversión en formato de punto flotante y punto fijo, en algunos casos sin pérdida de cuantización pero en otros con necesidad de aplicar truncamiento y/o saturación. Se estudiaron pocos casos de suma y producto en punto fijo pero con el objetivo de explorar varios tipos de representación, en este caso el *CSD*, *Booth radix-2* y *RNS*. En todos los casos, los modelos de *Python* y *Verilog* coincidieron perfectamente.

4. Referencias

- [1] E. Monja y G. R. D. Zúñiga, *Módulo 3 - Ejercicios*, Repositorio en GitHub, Diseño Digital para Sistemas DSP, Archivos desarrollados para los ejercicios del Módulo N°3, 2026. dirección: https://github.com/ErnestMonja/Diseno_Digital_para_Sistemas_DSP/tree/main/M%C3%B3dulo%203%20-%20Ejercicios.